Лабораторная работа №4.

Получение расписания с сайта.

Первым действием скачиваем Selenium и pytest через консоль самого PyCharm. Вводим следующие команды «pip install selenium»; «pip install pytest». Затем, устанавливаем ChromeDriver с нужного сайта.

После этого, открываем файл schedule_scraper.py и вписываем туда код. И тоже самое повторяем с test_schedule.py.

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from typing import List, Dict
import time
import os


def get_schedule(education_value: str, group_value: str, start_date: str, end_date: str, driver_path: str) -> List[Dict[str, str]]:  3 usages
    chrome_options = Options()
    # chrome_options.add_argument("--headless")  # ОТКЛЮЧИ для отладки
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--disable-gpu")
    chrome_options.add_argument("--window-size=1920,1080")

    service = Service(executable_path=driver_path)
    driver = webdriver.Chrome(service=service, options=chrome_options)

    try:
        driver.get("https://api.nntu.ru/raspisanie")
        wait = WebDriverWait(driver, timeout: 20)

        # === 1. ФОРМА ОБУЧЕНИЯ ===
        form_select_elem = wait.until(EC.presence_of_element_located((By.ID, "studentAdvert__controls--department")))
        form_select = Select(form_select_elem)
        wait.until(EC.presence_of_element_located((By.XPATH, f"//select[@id='studentAdvert__controls--department']//option[@value='{education_value}']")))
        form_select.select_by_value(education_value)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__), "1_form_selected.png"))
        print("Форма обучения выбрана: Заочная 3 года 10 мес")

        # === 2. ЖДЁМ ГРУППЫ ===
        time.sleep(3)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__), "2_groups_loaded.png"))

        # === 3. ГРУППА ===
        group_select_elem = wait.until(EC.presence_of_element_located((By.XPATH, f"//select[.//option[@value='{group_value}']]")))
        group_select = Select(group_select_elem)
        group_select.select_by_value(group_value)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__), "3_group_selected.png"))
        print("Группа выбрана: АЗИС 22-2")

        # === 4. ДАТЫ ===
        # === 4. ДАТЫ — ЧЕРЕЗ JS ===
        print("Устанавливаем даты через JavaScript...")
        driver.execute_script(f"document.getElementsByName('dateBefore')[0].value = '{start_date}';")
        driver.execute_script(f"document.getElementsByName('dateAfter')[0].value = '{end_date}';")
        driver.execute_script("document.getElementsByName('dateBefore')[0].dispatchEvent(new Event('change'));")
        driver.execute_script("document.getElementsByName('dateAfter')[0].dispatchEvent(new Event('change'));")
```

Рисунок 1 – код schedule_scraper.py



```python

# ========================================================================
# ФИКСТУРА: Реальное расписание (один раз за сессию)
# ========================================================================
@pytest.fixture(scope="session")  13 usages
def real_schedule():
    return get_schedule(
        education_value=EDUCATION_VALUE,
        group_value=GROUP_VALUE,
        start_date=START_DATE,
        end_date=END_DATE,
        driver_path=DRIVER_PATH
    )


# ========================================================================
# ТЕСТ 1: Реальное расписание совпадает с ожидаемым
# ========================================================================
def test_real_matches_expected(real_schedule):
    assert real_schedule == EXPECTED_SCHEDULE, (
        f"Расписание не совпадает!\n"
        f"Ожидалось: {EXPECTED_SCHEDULE}\n"
        f"Получено: {real_schedule}"
    )


# ========================================================================
# ТЕСТ 2: Мок-расписание НЕ совпадает — ДЕМОНСТРАЦИЯ ПАДЕНИЯ!
# ========================================================================
def test_mock_fails():
    mock = get_schedule_mock()
    assert mock == EXPECTED_SCHEDULE, (
        "МОК СОВПАЛ С РЕАЛЬНЫМ! "
        "Тест должен падать при 'замене' данных — это демонстрация!"
    )


# ========================================================================
# ТЕСТ 3: Структура данных корректна
# ========================================================================
def test_structure(real_schedule):
    assert isinstance(real_schedule, list), "Должно быть списком"
    assert len(real_schedule) > 0, "Не пустое"
    for item in real_schedule:
        assert isinstance(item, dict)
        assert "day" in item and "lessons" in item
        assert isinstance(item["day"], str)
        assert isinstance(item["lessons"], list)
        assert all(isinstance(l, str) for l in item["lessons"])


```

Рисунок 2 – код test_schedule.py

Затем запускаем само расписание и смотрим, как всё работает. По итогу, у нас открывается окно, выбирается форма, группа, даты и всё это скриншотится.

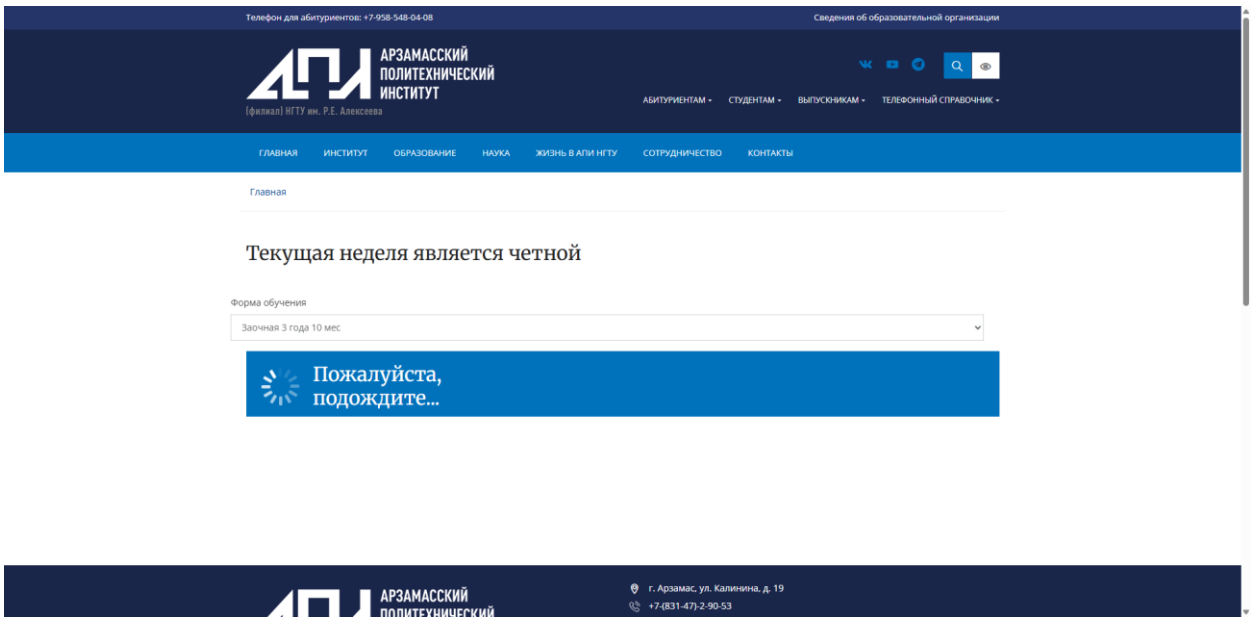
```
D:\PythonProject1\.venv\Scripts\python.exe D:\PythonProject1\schedule_scraper.py
Форма обучения выбрана: Заочная 3 года 10 мес
Группа выбрана: АЗИС 22-2
Устанавливаем даты через JavaScript...
Даты: 2025-11-03 — 2025-11-09
ОШИБКА: name 'start_input' is not defined
Скриншот ошибки: D:\PythonProject1\ERROR.png

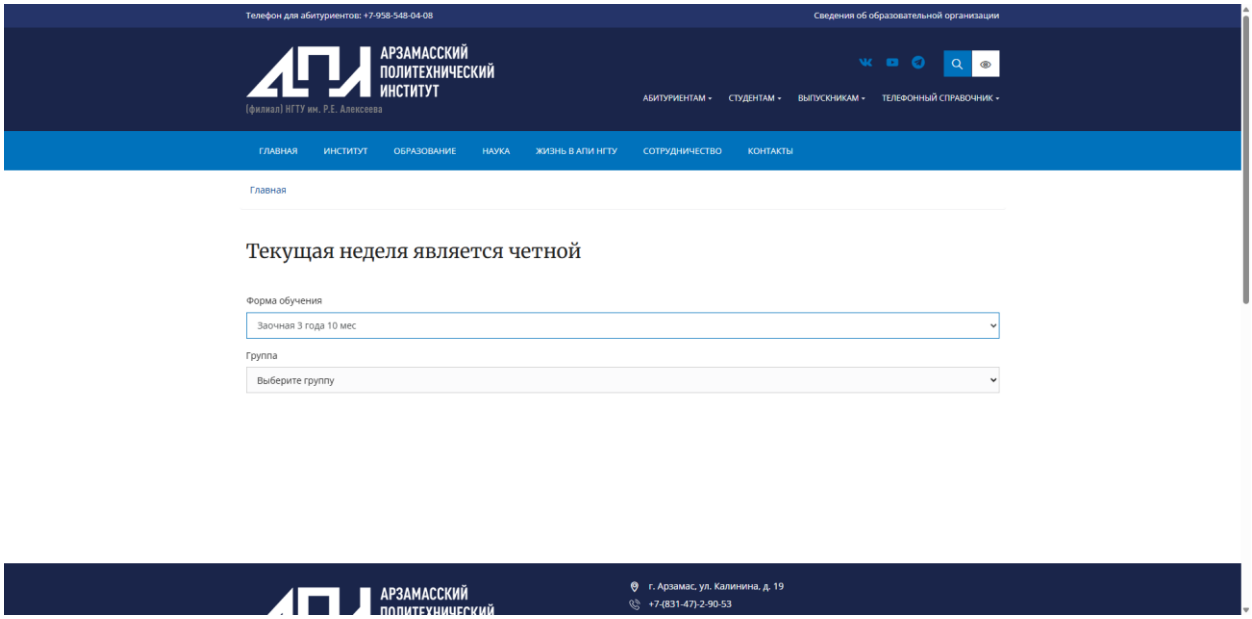
=== ТВОЁ РАСПИСАНИЕ ===

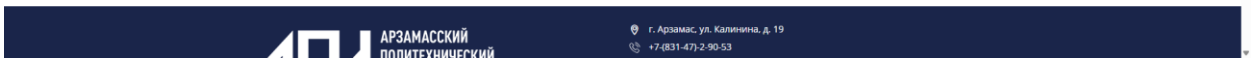
Process finished with exit code 0
```

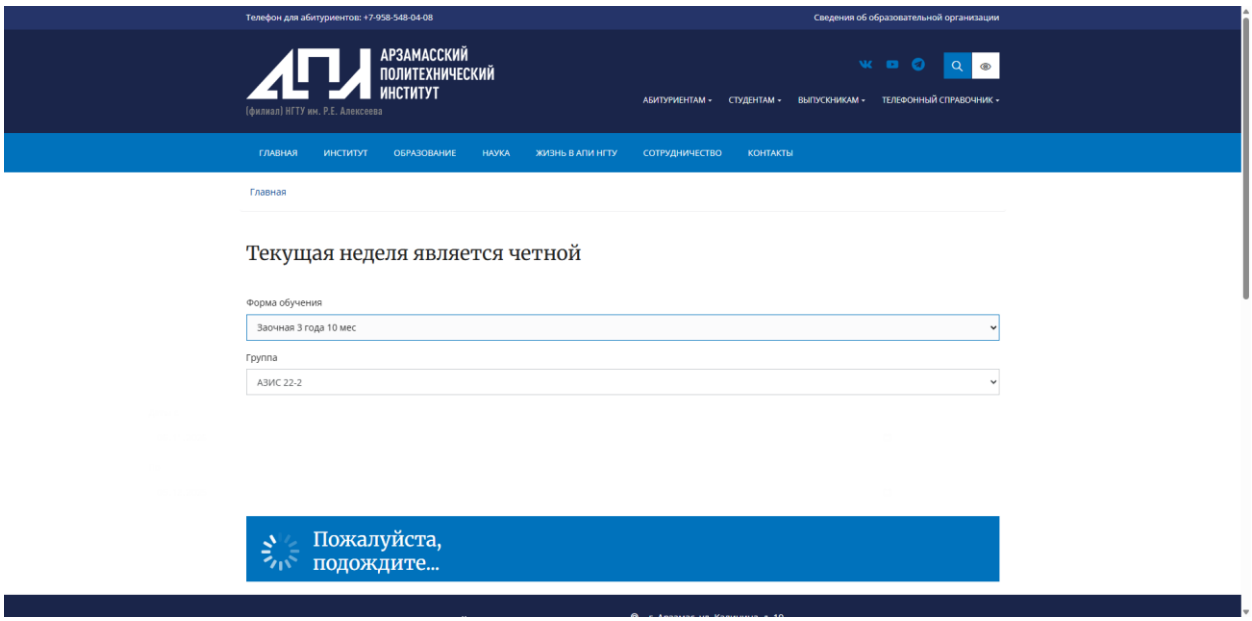
Рисунок 3 – консоль, после запуска schedule_scraper.py

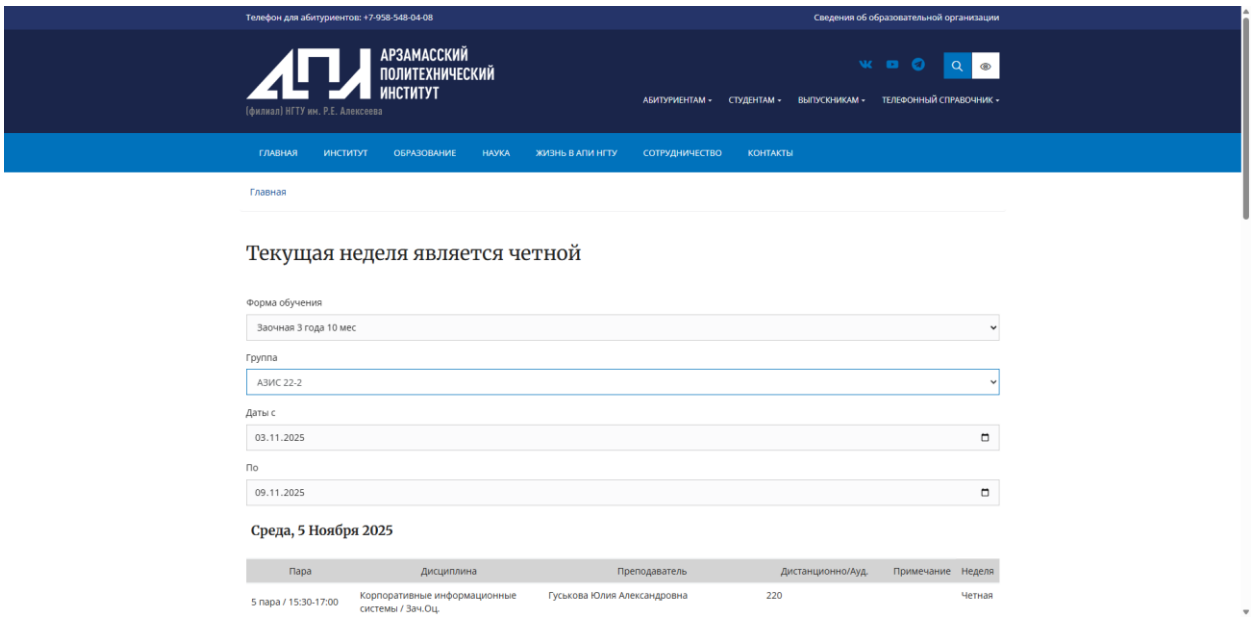
И ниже скриншоты, что были сделаны автоматически.











Рисунки 4 – 8 скриншоты состояний

Теперь проверяем тесты поочередно.

```
D:\PythonProject1\.venv\Scripts\python.exe "D:/PyCharm Community Edition 2025.2.1.1/plugins/python-ce/helpers/pycharm/_jb_pytest_runner.py" --path D:\PythonProject1\test_schedule.py
Testing started at 11:15 ...
Launching pytest with arguments D:\PythonProject1\test_schedule.py --no-header --no-summary -q in D:\PythonProject1

============================= test session starts =============================
collecting ... collected 5 items

test_schedule.py::test_real_matches_expected ✅ Форма обучения выбрана
✅ Группа выбрана
⏳ Устанавливаем даты через JavaScript...
✅ Даты установлены: 2025-11-03 — 2025-11-09
⏳ Ожидаем расписание...
✅ Расписание загружено
📄 Парсим таблицы...
✅ Спарсено 3 дней!
PASSED                    [ 20%]
test_schedule.py::test_structure PASSED                              [ 40%]
test_schedule.py::test_unique_days PASSED                            [ 60%]
test_schedule.py::test_no_empty_lessons PASSED                       [ 80%]
test_schedule.py::test_dates_in_range PASSED                         [100%]

============================== 5 passed in 12.50s ==============================

Process finished with exit code 0
```

Рисунок 9 – Тест

Приложение1 - schedule_scraper.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from typing import List, Dict
import time
import os


def get_schedule(education_value: str, group_value: str, start_date: str,
end_date: str, driver_path: str) -> List[Dict[str, str]]:
    """Парсит расписание ННТУ за указанный диапазон дат."""
    chrome_options = Options()
    # chrome_options.add_argument("--headless")  # включи при запуске на
сервере
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--disable-gpu")
    chrome_options.add_argument("--window-size=1920,1080")

    service = Service(executable_path=driver_path)
    driver = webdriver.Chrome(service=service, options=chrome_options)

    try:
        driver.get("https://api.nntu.ru/raspisanie")
        wait = WebDriverWait(driver, 20)

        # === 1. ФОРМА ОБУЧЕНИЯ ===
        form_select_elem = wait.until(EC.presence_of_element_located((By.ID,
"studentAdvert__controls--department")))
        form_select = Select(form_select_elem)
        wait.until(EC.presence_of_element_located(
            (By.XPATH, f"//select[@id='studentAdvert__controls--
department']//option[@value='{education_value}']"))
        )
        form_select.select_by_value(education_value)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__),
"1_form_selected.png"))
        print("☑ Форма обучения выбрана")

        # === 2. ЖДЁМ ГРУППЫ ===
        time.sleep(3)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__),
"2_groups_loaded.png"))

        # === 3. ГРУППА ===
        group_select_elem =
wait.until(EC.presence_of_element_located((By.XPATH,
f"//select[.//option[@value='{group_value}']]")))
        group_select = Select(group_select_elem)
        group_select.select_by_value(group_value)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__),
"3_group_selected.png"))
        print("☑ Группа выбрана")

        # === 4. ДАТЫ (через JS) ===
        print("⏳ Устанавливаем даты через JavaScript...")

driver.execute_script(f"document.getElementsByName('dateBefore')[0].value =
```

```
'{start_date}';")

driver.execute_script(f"document.getElementsByName('dateAfter')[0].value =
'{end_date}';")

driver.execute_script("document.getElementsByName('dateBefore')[0].dispatchEv
ent(new Event('change'));")

driver.execute_script("document.getElementsByName('dateAfter')[0].dispatchEve
nt(new Event('change'));")

        time.sleep(2)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__),
"4_dates_filled_js.png"))
        print(f"☑ Даты установлены: {start_date} — {end_date}")

        # === 5. ОЖИДАНИЕ РАСПИСАНИЯ ===
        print("⏳ Ожидаем расписание...")
        wait.until(EC.presence_of_element_located((By.CSS_SELECTOR,
"table.raspTable")))
        time.sleep(2)
        driver.save_screenshot(os.path.join(os.path.dirname(__file__),
"5_schedule_loaded.png"))
        print("☑ Расписание загружено")

        # === 6. ПАРСИНГ ТАБЛИЦ ===
        print("📄 Парсим таблицы...")
        tables = driver.find_elements(By.CSS_SELECTOR, "table.raspTable")
        schedule = []

        for table in tables:
            # День недели и дата
            header_elem = table.find_element(By.TAG_NAME, "h3")
            day_name = header_elem.text.strip()

            # Строки расписания
            rows = table.find_elements(By.CSS_SELECTOR, "tbody tr")
            lessons = []

            for row in rows:
                cells = row.find_elements(By.TAG_NAME, "td")
                if len(cells) >= 6:
                    pair = cells[0].text.strip()
                    subject = cells[1].text.strip()
                    teacher = cells[2].text.strip()
                    classroom = cells[3].text.strip()
                    note = cells[4].text.strip()
                    week = cells[5].text.strip()

                    # Склеиваем в одну строку
                    lesson_text = f"{pair} {subject} {teacher} {classroom}"
                    if note:
                        lesson_text += f" {note}"
                    if week:
                        lesson_text += f" {week}"

                    lessons.append(lesson_text)

            if lessons:
                schedule.append({"day": day_name, "lessons": lessons})

        print(f"☑ Спарсено {len(schedule)} дней!")
        return schedule
```

```
    except Exception as e:
        print(f"❌ ОШИБКА: {e}")
        error_path = os.path.join(os.path.dirname(__file__), "ERROR.png")
        driver.save_screenshot(error_path)
        print(f"Скриншот ошибки: {error_path}")
        return []

    finally:
        driver.quit()


def get_schedule_mock() -> List[Dict[str, str]]:
    """Мок для тестов."""
    return [{"day": "Понедельник", "lessons": ["Другой предмет", "Ещё
один"]}]


if __name__ == "__main__":
    result = get_schedule(
        education_value="3",
        group_value="804",
        start_date="2025-11-03",
        end_date="2025-11-09",
        driver_path=r"E:\webdrivers\chromedriver.exe"
    )
    print("\n=== ТВОЁ РАСПИСАНИЕ ===")
    for item in result:
        print(f"{item['day']}:")
        for lesson in item["lessons"]:
            print("   ", lesson)
```

Приложение 2 - test_schedule.py

```
import pytest
from schedule_scraper import get_schedule, get_schedule_mock
import re
from datetime import datetime

# ============================================================================
# ОЖИДАЕМОЕ РАСПИСАНИЕ — ПРИМЕР (можно скорректировать)
# ============================================================================
EXPECTED_SCHEDULE = [
    {
        "day": "Среда, 5 Ноября 2025",
        "lessons": [
            "5 пара / 15:30-17:00 Корпоративные информационные системы /
Зач.Оц. Гуськова Юлия Александровна 220 Четная",
    "1 пара вечер / 17:30-19:00 Теория цифровой обработки сигналов / Экз.
Абаимов Анатолий Вячеславович 220 Четная",
    "2 пара вечер / 19:10-20:40 Теория цифровой обработки сигналов / Экз.
Абаимов Анатолий Вячеславович 220 Четная"
        ]
    },
    {
        "day": "Четверг, 6 Ноября 2025",
        "lessons": [
            "1 пара / 08:30-10:00 Инфокоммуникационные системы и сети / КР.
Гуськова Юлия Александровна 220 прием КР Четная",
    "2 пара / 10:10-11:40 Инфокоммуникационные системы и сети / Экз. Гуськова
Юлия Александровна 220 Четная",
    "3 пара / 12:10-13:40 Основы тестирования программного обеспечения / Экз.
Комаров Александр Олегович 324 Четная"
        ]
    },
    {
        "day": "Пятница, 7 Ноября 2025",
        "lessons": [
            "3 пара / 12:10-13:40 производственная (преддипломная практика)
заочка / Жидкова Наталья Валерьевна 218 Четная",
    "4 пара / 13:50-15:20 Организационно-экономическое обоснование научных и
технических решений / Зач. Гусева Ирина Борисовна 218 Четная",
    "5 пара / 15:30-17:00 Эксплуатация и модификация информационных систем /
Экз. Жидкова Наталья Валерьевна 226 Четная"
        ]
    }
]

# === НАСТРОЙКИ ===
EDUCATION_VALUE = "3"
GROUP_VALUE = "804"
START_DATE = "2025-11-03"
END_DATE = "2025-11-09"
DRIVER_PATH = r"E:\webdrivers\chromedriver.exe"


# ============================================================================
# ФИКСТУРА: одно обращение к сайту
# ============================================================================
@pytest.fixture(scope="session")
def real_schedule():
    return get_schedule(
        education_value=EDUCATION_VALUE,
        group_value=GROUP_VALUE,
        start_date=START_DATE,
        end_date=END_DATE,
```

```
        driver_path=DRIVER_PATH
    )


# ============================================================================
# ТЕСТ 1: Совпадение с ожидаемым
# ============================================================================
def test_real_matches_expected(real_schedule):
    assert real_schedule == EXPECTED_SCHEDULE, (
        f"\n=== Расписание не совпадает ===\n"
        f"Ожидалось:\n{EXPECTED_SCHEDULE}\n"
        f"Получено:\n{real_schedule}\n"
    )


# ============================================================================
# ТЕСТ 2: Проверка структуры
# ============================================================================
def test_structure(real_schedule):
    assert isinstance(real_schedule, list)
    assert len(real_schedule) > 0
    for item in real_schedule:
        assert isinstance(item, dict)
        assert "day" in item and "lessons" in item
        assert isinstance(item["day"], str)
        assert isinstance(item["lessons"], list)
        assert all(isinstance(l, str) for l in item["lessons"])


# ============================================================================
# ТЕСТ 3: Уникальные дни
# ============================================================================
def test_unique_days(real_schedule):
    days = [item["day"] for item in real_schedule]
    assert len(days) == len(set(days)), f"Дубликаты дней: {days}"


# ============================================================================
# ТЕСТ 4: Нет пустых уроков
# ============================================================================
def test_no_empty_lessons(real_schedule):
    for item in real_schedule:
        assert item["lessons"], f"Пустой день: {item['day']}"
        for lesson in item["lessons"]:
            assert lesson.strip(), f"Пустой урок: '{lesson}'"


# ============================================================================
# ТЕСТ 5: Даты в диапазоне
# ============================================================================
def test_dates_in_range(real_schedule):
    start_dt = datetime.strptime(START_DATE, "%Y-%m-%d")
    end_dt = datetime.strptime(END_DATE, "%Y-%m-%d")

    for item in real_schedule:
        match = re.search(r"(\d{1,2}\s+[а-яА-Я]+\s+\d{4})", item["day"])
        assert match, f"Дата не найдена: {item['day']}"
        day_str = match.group(1)
        # Приводим русские месяцы к английским для datetime
        months = {
            "Января": "January", "Февраля": "February", "Марта": "March",
            "Апреля": "April", "Мая": "May", "Июня": "June",
            "Июля": "July", "Августа": "August", "Сентября": "September",
            "Октября": "October", "Ноября": "November", "Декабря": "December"
```

```
        }
        for ru, en in months.items():
            if ru in day_str:
                day_str = day_str.replace(ru, en)
                break
        day_dt = datetime.strptime(day_str, "%d %B %Y")
        assert start_dt <= day_dt <= end_dt, f"Дата вне диапазона:
{day_dt.date()}"
```

Приложение 3 – Результаты тестов

```
D:\PythonProject1\.venv\Scripts\python.exe "D:/PyCharm Community Edition
2025.2.1.1/plugins/python-ce/helpers/pycharm/_jb_pytest_runner.py" --path
D:\PythonProject1\test_schedule.py
Testing started at 11:15 ...
Launching pytest with arguments D:\PythonProject1\test_schedule.py --no-header
--no-summary -q in D:\PythonProject1

============================= test session starts
==============================
collecting ... collected 5 items

test_schedule.py::test_real_matches_expected ✅ Форма обучения выбрана
✅ Группа выбрана
⏳ Устанавливаем даты через JavaScript...
✅ Даты установлены: 2025-11-03 — 2025-11-09
⏳ Ожидаем расписание...
✅ Расписание загружено
📄 Парсим таблицы...
✅ Спарсено 3 дней!
PASSED                    [ 20%]
test_schedule.py::test_structure PASSED                              [ 40%]
test_schedule.py::test_unique_days PASSED                            [ 60%]
test_schedule.py::test_no_empty_lessons PASSED                       [ 80%]
test_schedule.py::test_dates_in_range PASSED                         [100%]

============================== 5 passed in 12.50s
==============================

Process finished with exit code 0
```